

Log & Trace Feature Engineering

Execution Spec for feature_engineering_node — implement directly from this document

0. Scope and Where This Runs

This spec covers the two functions inside `feature_engineering_node` that turn raw log lines and raw trace spans into numeric feature columns for the classifier. Node sits directly after `collect_node`, before `cluster_node`. Metrics feature engineering (P99-P50, growth rates) is out of scope for this document — this covers only the log path and the trace path.

Function	Consumes Produces
<code>engineer_log_features()</code>	raw log lines for this cycle
<code>engineer_trace_features()</code>	raw spans for this cycle

1. Log Feature Engineering

1.1 Offline Prerequisite — Train Drain Once

Before this node can run, Drain must be trained offline on the full corpus of log lines produced by the 13 known failure modes. This produces a frozen template registry, versioned as an artifact alongside `weibull_params.json` and `hmm_matrices.pkl`.

```
# offline/train_log_templates.py
from drain3 import TemplateMiner
from drain3.template_miner_config import TemplateMinerConfig

config = TemplateMinerConfig()
config.load("drain3.ini")
miner = TemplateMiner(config=config)

for log_line in training_corpus_all_13_modes:
    miner.add_log_message(log_line["log_message"])

known_template_ids = {c.cluster_id for c in miner.drain.clusters}

json.dump(list(known_template_ids), open("known_log_templates.json", "w"))
miner.save_state() # persist Drain's internal state -> drain_state.bin
```

Output artifacts: `known_log_templates.json`, `drain_state.bin`. Both are loaded read-only by the online node below — never retrained inline.

1.2 Input — Raw Log Line Schema

Exactly as produced by the generator / `collect_node`, per log line:

```
{
  "episode_id":    "ep_00042",
  "failure_mode":  "MEMORY_LEAK",    // present in training data; ABSENT in live/sim
stream
  "service":       "order-service",
  "elapsed_s":     180.0,
  "timestamp":     1719760222.0,
  "log_level":     "CRITICAL",
  "exception_type": null,
```

```

    "log_message":      "Major GC pause=185ms heap_mb=820 memory_utilization=65%"
}

```

1.3 Function Signature

```

def engineer_log_features(
    log_lines_this_cycle: list[dict],
    template_miner: TemplateMiner,      # loaded once at graph startup
    known_template_ids: set[int],       # loaded once at graph startup
) -> dict:
    ...

```

1.4 Output — Exact JSON Shape (this is what ships to the classifier)

```

{
  "log_count":          1,
  "log_max_severity":   4,
  "log_critical_count": 1,
  "log_has_exception":  0,
  "log_has_novel_template": 0,

  "_log_template_ids_seen": [7],
  "_log_raw_lines":        ["Major GC pause=185ms heap_mb=820 memory_utilization=65%"]
}

```

Fields WITHOUT the underscore prefix go into the classifier's feature vector. Fields WITH the underscore prefix (_log_template_ids_seen, _log_raw_lines) are carried alongside the feature vector through the graph but are NEVER passed to the classifier — they exist only for diagnose_node / generate_postmortem_node to quote as evidence later. Do not flatten or drop them before that point.

Field reference

Field	Type / Range
log_count	int, >= 0
log_max_severity	int, 0-4
log_critical_count	int, >= 0
log_has_exception	0 or 1
log_has_novel_template	0 or 1
_log_template_ids_seen	list[int]
_log_raw_lines	list[str]

Explicitly excluded from this feature set

Do NOT add keyword flags such as log_has_oom_keyword, log_has_deploy_keyword, log_has_timeout_keyword to the classifier input. These restate the label rather than providing independent evidence (a log line containing the word "deploy" is close to stating the answer for BAD_DEPLOY directly). If keyword detection is wanted for narration, compute it separately and store it under an underscore-prefixed evidence field, never as a numbered feature column.

1.5 Worked I/O Example — Normal MEMORY_LEAK Cycle

```

INPUT:
[{"log_level": "CRITICAL", "exception_type": null,
  "log_message": "Major GC pause=185ms heap_mb=820 memory_utilization=65%"}]

```

```

Drain match: template_id=7, matches "known MEMORY_LEAK" template -> in registry

```

OUTPUT:

```
{
  "log_count": 1, "log_max_severity": 4, "log_critical_count": 1,
  "log_has_exception": 0, "log_has_novel_template": 0,
  "_log_template_ids_seen": [7],
  "_log_raw_lines": ["Major GC pause=185ms heap_mb=820 memory_utilization=65%"]
}
```

1.6 Worked I/O Example — Genuinely Novel Log Line

INPUT:

```
[{"log_level": "ERROR", "exception_type": "CircuitBreakerException",
  "log_message": "circuit breaker tripped: pool exhausted after 3 retries"}]
```

Drain match: no cluster within similarity threshold -> NEW template_id=41 created,
41 NOT in known_template_ids

OUTPUT:

```
{
  "log_count": 1, "log_max_severity": 3, "log_critical_count": 0,
  "log_has_exception": 1, "log_has_novel_template": 1,
  "_log_template_ids_seen": [41],
  "_log_raw_lines": ["circuit breaker tripped: pool exhausted after 3 retries"]
}
```

log_has_novel_template=1 here is the deterministic trigger, combined with low classifier confidence and high HMM posterior entropy (see Section 3), for routing this cycle to the UNKNOWN path.

2. Trace Feature Engineering — Petri Net Conformance

2.1 Offline Prerequisite — Discover One Process Model Per Failure Mode

For each of the 13 failure modes, discover a process model from that mode's training traces using PM4Py's Inductive Miner. This produces one Petri net per mode, capturing the legal span orderings, branching, and concurrency observed during training — not just one fixed shape.

```
# offline/discover_trace_models.py
import pm4py

process_models = {} # mode -> (Petri net, initial_marking, final_marking)

for mode in ALL_13_MODES:
    event_log = build_event_log(training_traces_for_mode(mode))
    # event_log format: one "case" per trace_id, one "event" per span,
    # with span_name as activity and start_time as timestamp
    net, im, fm = pm4py.discover_petri_net_inductive(event_log)
    process_models[mode] = (net, im, fm)
    pm4py.write_pnml(net, im, fm, f"process_models/{mode}.pnml")
```

Output artifact: 13 .pnml files, one per mode, versioned alongside the other offline artifacts. These are loaded read-only online — never rediscovered inline.

2.2 Input — Raw Span Schema

```
{
  "episode_id":      "ep_00042",
  "failure_mode":    "MEMORY_LEAK",    // ABSENT in live/sim stream
  "trace_id":        "a1b2c3d4e5f6",
  "span_id":         "sp001",
  "parent_span_id":  null,              // null = root span
  "span_name":       "HTTP request",
  "span_duration_ms": 320.4,
  "span_status":     "OK",
  "service_version": "v1.0.0",
  "peer_service":    null,
  "db_operation_type": null,
  "elapsed_s":       180.0,
  "timestamp":       1719760222.0
}
```

2.3 Function Signature

```
def engineer_trace_features(
    spans_this_cycle: list[dict],
    process_models: dict[str, tuple], # mode -> (net, im, fm), loaded at startup
    conformance_threshold: float = 0.5,
) -> dict:
    ...
```

2.4 Conformance Scoring — What Actually Runs

For each distinct trace_id observed this cycle, build a PM4Py event log fragment from its spans, then run token-based replay (or alignment-based conformance for a stricter score) against EACH of the 13 stored process models. This produces 13 scores per trace, one per mode, in the 0-1 range where 1.0 means perfect fit.

```
def conformance_against_all_modes(trace_spans, process_models):
    trace_event_log = spans_to_event_log(trace_spans)
    scores = {}
```

```

for mode, (net, im, fm) in process_models.items():
    result = pm4py.conformance_diagnostics_token_based_replay(
        trace_event_log, net, im, fm
    )
    scores[mode] = result[0]["trace_fitness"]    # 0.0 - 1.0
return scores

```

Aggregate across all traces in the cycle by taking the mean conformance score per mode. This produces 13 numeric columns — `trace_conformance_MEMORY_LEAK`, `trace_conformance_CPU_SATURATION`, etc. — which go into the classifier alongside the existing structural features (gap ratio, uniform slowdown score, duplicate trace ID count).

2.5 Output — Exact JSON Shape (this is what ships to the classifier)

```

{
  "trace_count": 1,
  "root_span_duration_mean": 320.4,
  "root_span_error_rate": 0.0,
  "span_gap_ratio": 0.61,
  "uniform_slowdown_score": 0.08,
  "duplicate_trace_id_count": 0,
  "distinct_error_services": 0,
  "write_span_duration_mean": 0.0,
  "read_span_duration_mean": 0.0,

  "trace_conformance_NONE": 0.12,
  "trace_conformance_MEMORY_LEAK": 0.94,
  "trace_conformance_CPU_SATURATION": 0.31,
  "trace_conformance_LATENCY_SPIKE": 0.71,
  "trace_conformance_ERROR_STORM": 0.05,
  "trace_conformance_DB_SLOWDOWN": 0.18,
  "trace_conformance_CACHE_STAMPEDE": 0.15,
  "trace_conformance_QUEUE_BACKUP": 0.10,
  "trace_conformance_DEPENDENCY_TIMEOUT": 0.09,
  "trace_conformance_BAD_DEPLOY": 0.07,
  "trace_conformance_RETRY_STORM": 0.11,
  "trace_conformance_DISK_IO_SATURATION": 0.06,
  "trace_conformance_CASCADING_FAILURE": 0.20,

  "trace_has_novel_shape": 0,

  "_trace_ids_this_cycle": ["a1b2c3d4e5f6"],
  "_slow_span_detail": [{"span_name": null, "peer_service": null,
    "duration_ms": null}]
}

```

Same convention as logs: underscore-prefixed fields never reach the classifier. `_slow_span_detail` specifically carries forward the `peer_service` name for any anomalous span, so `diagnose_node` can name the exact failing dependency in its narrative even though the classifier itself never sees that string.

Field reference

Field	Type / Range
<code>trace_count</code>	int, ≥ 0
<code>root_span_duration_mean</code>	float, ms
<code>root_span_error_rate</code>	float, 0-1
<code>span_gap_ratio</code>	float, 0-1
<code>uniform_slowdown_score</code>	float, 0-1

duplicate_trace_id_count	int, >= 0
distinct_error_services	int, >= 0
write_span_duration_mean / read_span_duration_mean	float, ms
trace_conformance_<MODE>	float, 0-1, x13
trace_has_novel_shape	0 or 1
_trace_ids_this_cycle	list[str]
_slow_span_detail	list[dict]

2.6 Worked I/O Example — Normal MEMORY_LEAK Cycle

```
INPUT (3 spans, one trace):
root:      duration=320.4ms, status=OK
db.query:  duration=15ms,   status=OK, parent=root
cache.get: duration=2ms,    status=OK, parent=root
```

Conformance: fits MEMORY_LEAK Petri net well (0.94) because the model was trained on traces with exactly this gap-in-root pattern. Fits LATENCY_SPIKE model reasonably too (0.71) since they share the same gap structure -- this partial-fit overlap is EXPECTED and is resolved by the metrics features (heap_growth_rate), not by trace conformance alone.

```
OUTPUT: see 2.5 above (this IS that example)
max(trace_conformance_*) = 0.94 -> trace_has_novel_shape = 0
```

2.7 Worked I/O Example — Genuinely Novel Trace Shape

```
INPUT (parallel error spans, a shape never seen in training):
root:      duration=890ms, status=ERROR
grpc.streaming_call: duration=780ms, status=ERROR, parent=root
grpc.streaming_call: duration=810ms, status=ERROR, parent=root (same span_name twice)
```

Conformance against all 13 models: every score below 0.4 -- this span structure and span_name do not appear in ANY training process model.

```
OUTPUT (abbreviated):
{
  "trace_conformance_NONE": 0.02, "trace_conformance_MEMORY_LEAK": 0.11,
  ... (all 13 scores low) ...,
  "trace_conformance_CASCADING_FAILURE": 0.38,

  "trace_has_novel_shape": 1,

  "_slow_span_detail": [
    {"span_name": "grpc.streaming_call", "peer_service": null, "duration_ms": 780},
    {"span_name": "grpc.streaming_call", "peer_service": null, "duration_ms": 810}
  ]
}
```

trace_has_novel_shape=1 combined with log_has_novel_template=1 (if the corresponding log line is also unmatched) and low classifier top-probability is the three-way corroborating signal for the UNKNOWN routing path — see Section 3.

3. How These Outputs Combine Downstream

Both functions run inside the same `feature_engineering_node`, alongside the existing metrics feature engineering. All non-underscore fields from both outputs are merged into ONE flat feature vector — this is what LightGBM receives. The underscore-prefixed evidence fields are merged into a separate evidence dict that travels alongside the feature vector through `cluster_node`, the classifier, and the HMM smoother, untouched, for `diagnose_node` to consume later.

```
def feature_engineering_node(state: AgentState) -> AgentState:
    metric_features = engineer_metric_features(state["raw_metrics"])
    log_features    = engineer_log_features(state["raw_logs"], template_miner,
known_template_ids)
    trace_features  = engineer_trace_features(state["raw_spans"], process_models)

    classifier_input = {
        **metric_features,
        **{k: v for k, v in log_features.items() if not k.startswith("_")},
        **{k: v for k, v in trace_features.items() if not k.startswith("_")},
    }
    evidence = {
        **{k: v for k, v in log_features.items() if k.startswith("_")},
        **{k: v for k, v in trace_features.items() if k.startswith("_")},
    }
    return {**state, "classifier_input": classifier_input, "evidence": evidence}
```

3.1 UNKNOWN Routing Rule

Route to the UNKNOWN state when ALL THREE of the following hold simultaneously — not any single one alone:

- 1. `log_has_novel_template == 1` (or no log line present this cycle)
- 2. `trace_has_novel_shape == 1`
- 3. `max(classifier softmax probabilities) < 0.35` AND `HMM posterior entropy ratio > 0.7`

Requiring all three avoids false UNKNOWN triggers from a single noisy signal — for instance, a missing log line on an otherwise clean, well-classified cycle should not alone force an UNKNOWN routing.

3.2 Artifacts This Section Depends On

Artifact	Produced by
known_log_templates.json	offline/train_log_templates.py
drain_state.bin	offline/train_log_templates.py
process_models/ <MODE>.pnml (x13)	offline/discover_trace_models.py
classifier.pkl	offline classifier training (out of scope here)

3.3 Open Items for Implementation

- Tune Drain's similarity threshold against a held-out validation set before freezing `known_log_templates.json` — too loose merges genuinely distinct templates, too strict floods `log_has_novel_template` with false positives.
- Tune `conformance_threshold` (default 0.5 above) the same way, against held-out traces from all 13 modes plus a small set of deliberately synthetic novel shapes.
- Token-based replay is faster but less precise than alignment-based conformance checking; benchmark both on your actual trace volume before committing to one for the online path.

- Confirm PM4Py's `discover_petri_net_inductive` produces a sound model (no deadlocks) for each of the 13 modes' training traces before freezing the .pnml files — Inductive Miner guarantees this by construction, but validate on your actual generated data as a sanity check.